OPPE 1

200 Most Important Questions

With Complete Solutions

- **Section A** - Data Types & Basics
- **Section B** - Data Processing
- **Section C** - I/O Based Problem Solving

Section A: Data Types & Basics

Function-based questions. Implement the function. No input/output handling needed.

Q1. [PYQ] abs_diff_between_sum_and_sum_of_squares

EASY

Given two integers, find the absolute difference between their sum and the sum of their squares. Example: a=2, b=3 ⇒ 8

```
def abs_diff_between_sum_and_sum_of_squares(a: int, b: int) -> int:
    pass
```

Answer:

```
def abs_diff_between_sum_and_sum_of_squares(a: int, b: int) -> int:
    return abs((a + b) - (a**2 + b**2))
```

Q2. [PYQ] swap_except_middle_three

HARD

Given an odd-length string (at least 5 chars), swap parts before and after the middle three characters. Example: "abcdefghi" ⇒ "ghidefabc"

Answer:

```
def swap_except_middle_three(s: str) -> str:
    mid = len(s) // 2
    before = s[:mid-1]
    middle = s[mid-1:mid+2]
    after = s[mid+2:]
    return after + middle + before
```

Q3. [PYQ] interleave_lists

MEDIUM

Given three lists of the same length, interleave them. Example: [1,2], ['a','b'], [(1,1),(2,2)] ⇒ [1,'a',(1,1),2,'b',(2,2)]

Answer:

```
def interleave_lists(list1, list2, list3):
    result = []
    for a, b, c in zip(list1, list2, list3):
        result.extend([a, b, c])
    return result
```

Q4. [PYQ] has_more_than_5_unique_digits

EASY

Determine if a given integer has more than 5 unique digits. Example: 1234567 ⇒ True

Answer:
```python
def has_more_than_5_unique_digits(num: int) -> bool:
    return len(set(str(abs(num)))) > 5
```

Q5. [PYQ] final_position

EASY

Given position (x1,y1), velocity (vx,vy), and time t, return final position tuple.

Answer:
```python
def final_position(pos: tuple, vel: tuple, time: int) -> tuple:
    return (pos[0] + vel[0] * time, pos[1] + vel[1] * time)
```

Q6. [PYQ] remove_keys_not_in_list

MEDIUM

Remove keys from a dictionary not present in a given list. Modify in place. Make a copy of keys before iterating.

Answer:
```python
def remove_keys_not_in_list(d: dict, l: list) -> None:
    keys_to_remove = [k for k in d if k not in l]
    for k in keys_to_remove:
        del d[k]
```

Q7. [PYQ] is_positive_odd_or_negative_even

EASY

Check if integer is positive odd or negative even. Examples: 3⇒True, −4⇒True, 4⇒False, 0⇒False

Answer:
```python
def is_positive_odd_or_negative_even(n: int) -> bool:
    if n > 0 and n % 2 != 0:
        return True
    if n < 0 and n % 2 == 0:
        return True
    return False
```

Q8. [PYQ] replace_middle_with_n_times

HARD

Replace middle element of odd-length tuple with n copies. Example: (1,2,3,4,5), n=3 ⇒ (1,2,3,3,3,4,5)

Answer:
```python
def replace_middle_with_n_times_middle(t: tuple, n: int) -> tuple:
    mid = len(t) // 2
    return t[:mid] + (t[mid],) * n + t[mid+1:]
```

Q9. [PYQ] div_by_exactly_one

EASY

Check if num is divisible by exactly one of a or b. Example: (20,5,6)⇒True, (20,5,10)⇒False

Answer:

```
def div_by_exactly_one(num: int, a: int, b: int) -> bool:
    return (num % a == 0) != (num % b == 0)
```

The != on booleans acts as XOR.

Q10. [PYQ] mm_dd_yy_to_yy_dd_mm

EASY

Convert date format. Example: "12-25-21" ⇒ "21-25-12"

Answer:

```
def mm_dd_yy_to_yy_dd_mm(date: str) -> str:
    parts = date.split("-")
    return f"{parts[2]}-{parts[1]}-{parts[0]}"
```

Q11. [PYQ] increment_value_with_max_limit

MEDIUM

Increment dict value by inc, cap at limit. Modify in place.

Answer:

```
def increment_value_with_max_limit(d: dict, key: str, inc: int, limit: int) -> None:
    d[key] = min(d[key] + inc, limit)
```

Q12. [PYQ] is_right_triangle_with_even_sides

MEDIUM

Check if three sides form a right triangle where perpendicular sides are even.

Answer:

```
def is_right_triangle_with_even_sides(a: int, b: int, c: int) -> bool:
    sides = sorted([a, b, c])
    if sides[0]**2 + sides[1]**2 != sides[2]**2:
        return False
    return sides[0] % 2 == 0 and sides[1] % 2 == 0
```

Q13. [PYQ] is_odd_indices_alpha_even_indices_digits

MEDIUM

Check if odd-indexed chars are alphabets and even-indexed chars are digits.

Answer:

```
def is_odd_indices_alpha_and_even_indices_digits(s: str) -> bool:
    for i, ch in enumerate(s):
        if i % 2 == 0 and not ch.isdigit():
            return False
        if i % 2 == 1 and not ch.isalpha():
```

```
            return False
    return True
```

Q14. [PYQ] unique_chars_in_first_not_second

EASY

Return unique chars in s1 but not s2. Example: "water","watch" ⇒ {'e','r'}

Answer:

```
def unique_chars_present_in_first_not_in_second(s1: str, s2: str) -> set:
    return set(s1) - set(s2)
```

Q15. [PYQ] swap_even_and_odd_indices

MEDIUM

Swap values at even and odd indices in place. Example: [1,2,3,4] ⇒ [2,1,4,3]

Answer:

```
def swap_even_and_odd_indices(l: list) -> None:
    for i in range(0, len(l) - 1, 2):
        l[i], l[i+1] = l[i+1], l[i]
```

Q16. [PYQ] repeat_tuple

EASY

Given (a,b), return tuple with a repeated b times then b repeated a times. Example: (2,3) ⇒ (2,2,2,3,3)

Answer:

```
def repeat(t: tuple) -> tuple:
    a, b = t
    return (a,) * b + (b,) * a
```

Q17. [PYQ] num_squares

EASY

Dict with numbers 1 to n as keys and squares as values.

Answer:

```
def num_squares(n: int) -> dict:
    return {i: i**2 for i in range(1, n+1)}
```

Q18. [PYQ] within_and_has_double_quotes

HARD

Check if string starts/ends with double quote and has double quotes inside.

Answer:

```
def within_and_has_double_quotes(s: str) -> bool:
    if len(s) < 3:
        return False
```

```
    if s[0] != '"' or s[-1] != '"':
        return False
    return '"' in s[1:-1]
```

Q19. is_valid_triangle

EASY

Check if three sides can form a valid triangle.

Answer:

```
def is_valid_triangle(a: int, b: int, c: int) -> bool:
    return a + b > c and b + c > a and a + c > b
```

Q20. rotate_list_right

MEDIUM

Rotate list right by k positions in place. Example: [1,2,3,4,5], k=2 ⇒ [4,5,1,2,3]

Answer:

```
def rotate_list_right(l: list, k: int) -> None:
    k = k % len(l)
    l[:] = l[-k:] + l[:-k]
```

Q21. merge_alternating

MEDIUM

Merge two strings alternating chars. Example: "abc","12345" ⇒ "a1b2c345"

Answer:

```
def merge_alternating(s1: str, s2: str) -> str:
    result = []
    for i in range(max(len(s1), len(s2))):
        if i < len(s1): result.append(s1[i])
        if i < len(s2): result.append(s2[i])
    return "".join(result)
```

Q22. second_largest_in_tuple

EASY

Return second largest unique element. Example: (5,3,5,1,4) ⇒ 4

Answer:

```
def second_largest_in_tuple(t: tuple) -> int:
    return sorted(set(t), reverse=True)[1]
```

Q23. count_vowels_consonants

EASY

Return dict with "vowels" and "consonants" counts.

Answer:

```python
def count_vowels_consonants(s: str) -> dict:
    v = "aeiouAEIOU"
    vc = sum(1 for c in s if c.isalpha() and c in v)
    cc = sum(1 for c in s if c.isalpha() and c not in v)
    return {"vowels": vc, "consonants": cc}
```

Q24. flatten_nested_list

EASY

Flatten a 2D list. Example: [[1,2],[3,4],[5]] ⇒ [1,2,3,4,5]

Answer:

```python
def flatten_nested_list(lst: list) -> list:
    return [item for sublist in lst for item in sublist]
```

Q25. dict_from_two_lists

EASY

Create dict from list of keys and list of values.

Answer:

```python
def dict_from_two_lists(keys: list, values: list) -> dict:
    return dict(zip(keys, values))
```

Q26. reverse_words

MEDIUM

Reverse word order. Example: "hello world python" ⇒ "python world hello"

Answer:

```python
def reverse_words(s: str) -> str:
    return " ".join(s.split()[::-1])
```

Q27. is_palindrome

EASY

Check if string is a palindrome (case-insensitive).

Answer:

```python
def is_palindrome(s: str) -> bool:
    s = s.lower()
    return s == s[::-1]
```

Q28. sum_of_digits

EASY

Return sum of digits of a positive integer. Example: 1234 ⇒ 10

Answer:
```python
def sum_of_digits(n: int) -> int:
    return sum(int(d) for d in str(n))
```

Q29. clamp_value

EASY

Clamp value between lo and hi. Return lo if x<lo, hi if x>hi, else x.

Answer:
```python
def clamp(x: int, lo: int, hi: int) -> int:
    return max(lo, min(x, hi))
```

Q30. remove_duplicates_preserve_order

MEDIUM

Remove duplicates from list preserving first occurrence order.

Answer:
```python
def remove_duplicates(lst: list) -> list:
    seen = set()
    result = []
    for item in lst:
        if item not in seen:
            seen.add(item)
            result.append(item)
    return result
```

Q31. is_even_length_and_symmetric

MEDIUM

Check if string has even length and first half equals reverse of second half.

Answer:
```python
def is_even_length_and_symmetric(s: str) -> bool:
    if len(s) % 2 != 0:
        return False
    mid = len(s) // 2
    return s[:mid] == s[mid:][::-1]
```

Q32. tuple_element_frequency

MEDIUM

Return dict of element frequencies in a tuple.

Answer:
```python
def tuple_element_frequency(t: tuple) -> dict:
    freq = {}
    for item in t:
        freq[item] = freq.get(item, 0) + 1
    return freq
```

Q33. swap_first_last

EASY

Swap first and last elements of a list in place.

Answer:

```
def swap_first_last(l: list) -> None:
    if len(l) >= 2:
        l[0], l[-1] = l[-1], l[0]
```

Q34. string_to_list_of_words

EASY

Split string by spaces, remove empty strings. Example: " hello world " ⇒ ["hello","world"]

Answer:

```
def string_to_list_of_words(s: str) -> list:
    return s.split()
```

Q35. is_subset_dict

MEDIUM

Check if all keys of dict d1 exist in dict d2 with same values.

Answer:

```
def is_subset_dict(d1: dict, d2: dict) -> bool:
    for k, v in d1.items():
        if k not in d2 or d2[k] != v:
            return False
    return True
```

Q36. list_intersection

EASY

Return list of common elements (unique) between two lists.

Answer:

```
def list_intersection(l1: list, l2: list) -> list:
    return list(set(l1) & set(l2))
```

Q37. count_upper_lower_digit

EASY

Return dict with counts of "upper", "lower", "digit" in a string.

Answer:

```
def count_upper_lower_digit(s: str) -> dict:
    u = sum(1 for c in s if c.isupper())
    l = sum(1 for c in s if c.islower())
    d = sum(1 for c in s if c.isdigit())
    return {"upper": u, "lower": l, "digit": d}
```

Q38. merge_two_dicts

EASY

Merge two dicts. If key exists in both, sum the values.

Answer:

```
def merge_two_dicts(d1: dict, d2: dict) -> dict:
    result = d1.copy()
    for k, v in d2.items():
        result[k] = result.get(k, 0) + v
    return result
```

Q39. replace_vowels

EASY

Replace all vowels in string with given character. Example: "hello","*" ⇒ "h*ll*"

Answer:

```
def replace_vowels(s: str, ch: str) -> str:
    vowels = "aeiouAEIOU"
    return "".join(ch if c in vowels else c for c in s)
```

Q40. chunk_list

MEDIUM

Split list into chunks of size n. Example: [1,2,3,4,5], n=2 ⇒ [[1,2],[3,4],[5]]

Answer:

```
def chunk_list(lst: list, n: int) -> list:
    return [lst[i:i+n] for i in range(0, len(lst), n)]
```

Q41. is_anagram

MEDIUM

Check if two strings are anagrams (case-insensitive).

Answer:

```
def is_anagram(s1: str, s2: str) -> bool:
    return sorted(s1.lower()) == sorted(s2.lower())
```

Q42. dict_invert

MEDIUM

Invert a dictionary (swap keys and values). Assume values are unique.

Answer:

```
def dict_invert(d: dict) -> dict:
    return {v: k for k, v in d.items()}
```

Q43. first_non_repeating_char

HARD

Return first non-repeating character in a string. Example: "aabccbd" ⇒ "d"

Answer:

```
def first_non_repeating_char(s: str) -> str:
    freq = {}
    for c in s:
        freq[c] = freq.get(c, 0) + 1
    for c in s:
        if freq[c] == 1:
            return c
    return ""
```

Q44. matrix_diagonal_sum

EASY

Return sum of primary diagonal of a square matrix.

Answer:

```
def matrix_diagonal_sum(matrix: list) -> int:
    return sum(matrix[i][i] for i in range(len(matrix)))
```

Q45. zip_with_index

EASY

Return list of (index, element) tuples. Example: ['a','b'] ⇒ [(0,'a'),(1,'b')]

Answer:

```
def zip_with_index(lst: list) -> list:
    return list(enumerate(lst))
```

Q46. title_case

EASY

Capitalize first letter of each word. Example: "hello world" ⇒ "Hello World"

Answer:

```
def title_case(s: str) -> str:
    return s.title()
```

Q47. remove_all_occurrences

EASY

Remove all occurrences of a value from list in place.

Answer:

```
def remove_all_occurrences(l: list, val) -> None:
    while val in l:
        l.remove(val)
```

Q48. nested_dict_access

MEDIUM

Given nested dict and list of keys, return value. Example: {"a":{"b":5}}, ["a","b"] ⇒ 5

Answer:

```
def nested_dict_access(d: dict, keys: list):
    current = d
    for k in keys:
        current = current[k]
    return current
```

Q49. even_odd_partition

MEDIUM

Split list into two lists: evens and odds. Return (evens, odds).

Answer:

```
def even_odd_partition(lst: list) -> tuple:
    evens = [x for x in lst if x % 2 == 0]
    odds = [x for x in lst if x % 2 != 0]
    return (evens, odds)
```

Q50. running_sum

EASY

Return running sum of a list. Example: [1,2,3,4] ⇒ [1,3,6,10]

Answer:

```
def running_sum(lst: list) -> list:
    result = []
    total = 0
    for x in lst:
        total += x
        result.append(total)
    return result
```

Q51. string_compression

HARD

Compress string by counting consecutive chars. Example: "aaabbc" ⇒ "a3b2c1"

Answer:

```
def string_compression(s: str) -> str:
    if not s: return ""
    result = []
    count = 1
    for i in range(1, len(s)):
        if s[i] == s[i-1]:
            count += 1
        else:
            result.append(f"{s[i-1]}{count}")
            count = 1
```

```
    result.append(f"{s[-1]}{count}")
    return "".join(result)
```

Q52. max_consecutive_ones

MEDIUM

Return max consecutive 1s in a binary list. Example: [1,1,0,1,1,1] ⇒ 3

Answer:

```
def max_consecutive_ones(lst: list) -> int:
    max_c = 0
    curr = 0
    for x in lst:
        if x == 1:
            curr += 1
            max_c = max(max_c, curr)
        else:
            curr = 0
    return max_c
```

Q53. tuple_to_dict

EASY

Convert list of 2-element tuples to dict. Example: [('a',1),('b',2)] ⇒ {'a':1,'b':2}

Answer:

```
def tuple_to_dict(lst: list) -> dict:
    return dict(lst)
```

Q54. caesar_shift_char

MEDIUM

Shift a single character by k positions. Wrap around. Non-alpha unchanged.

Answer:

```
def caesar_shift_char(ch: str, k: int) -> str:
    if ch.isalpha():
        base = ord("A") if ch.isupper() else ord("a")
        return chr((ord(ch) - base + k) % 26 + base)
    return ch
```

Q55. list_difference

EASY

Return elements in first list but not in second.

Answer:

```
def list_difference(l1: list, l2: list) -> list:
    s2 = set(l2)
    return [x for x in l1 if x not in s2]
```

Q56. multiply_all_elements

EASY

Return product of all elements in a list.

Answer:

```
def multiply_all(lst: list) -> int:
    result = 1
    for x in lst:
        result *= x
    return result
```

Q57. deep_copy_nested_list

MEDIUM

Create a deep copy of a 2D list without using copy module.

Answer:

```
def deep_copy_2d(matrix: list) -> list:
    return [row[:] for row in matrix]
```

Q58. is_sorted

EASY

Check if list is sorted in non-decreasing order.

Answer:

```
def is_sorted(lst: list) -> bool:
    for i in range(len(lst) - 1):
        if lst[i] > lst[i+1]:
            return False
    return True
```

Q59. extract_digits

EASY

Extract only digits from a string. Example: "a1b2c3" ⇒ "123"

Answer:

```
def extract_digits(s: str) -> str:
    return "".join(c for c in s if c.isdigit())
```

Q60. most_frequent_element

MEDIUM

Return the most frequent element in a list.

Answer:

```
def most_frequent(lst: list):
    freq = {}
    for x in lst:
        freq[x] = freq.get(x, 0) + 1
```

```
    return max(freq, key=freq.get)
```

Q61. transpose_matrix

MEDIUM

Return transpose of a 2D matrix. Example: [[1,2],[3,4]] ⇒ [[1,3],[2,4]]

Answer:

```
def transpose(matrix: list) -> list:
    rows = len(matrix)
    cols = len(matrix[0])
    return [[matrix[i][j] for i in range(rows)] for j in range(cols)]
```

Q62. second_largest

EASY

Return the second largest element from a list of integers. Assume list has at least 2 distinct elements.

Answer:

```
def second_largest(lst: list) -> int:
    unique = list(set(lst))
    unique.sort()
    return unique[-2]
```

Q63. rotate_list

MEDIUM

Rotate a list to the right by k positions. Example: [1,2,3,4,5], k=2 ⇒ [4,5,1,2,3]

Answer:

```
def rotate_right(lst: list, k: int) -> list:
    k = k % len(lst)
    return lst[-k:] + lst[:-k]
```

Q64. is_anagram

MEDIUM

Check if two strings are anagrams (case-insensitive). Example: 'Listen', 'Silent' ⇒ True

Answer:

```
def is_anagram(s1: str, s2: str) -> bool:
    return sorted(s1.lower()) == sorted(s2.lower())
```

Q65. sum_of_digits

EASY

Return sum of digits of an integer. Example: 1234 ⇒ 10

Answer:

```
def sum_of_digits(n: int) -> int:
```

```
    return sum(int(d) for d in str(abs(n)))
```

Q66. chunk_list

MEDIUM

Split a list into chunks of size n. Last chunk may be smaller. Example: [1,2,3,4,5], n=2 ⇒ [[1,2],[3,4],[5]]

Answer:

```
def chunk_list(lst: list, n: int) -> list:
    return [lst[i:i+n] for i in range(0, len(lst), n)]
```

Q67. title_case

EASY

Convert string to title case (first letter of each word capitalized). Example: 'hello world' ⇒ 'Hello World'

Answer:

```
def title_case(s: str) -> str:
    return s.title()
```

Q68. flatten_list

MEDIUM

Flatten a nested list one level deep. Example: [[1,2],[3,[4]],5] ⇒ [1,2,3,[4],5]

Answer:

```
def flatten(lst: list) -> list:
    result = []
    for item in lst:
        if isinstance(item, list):
            result.extend(item)
        else:
            result.append(item)
    return result
```

Q69. remove_consecutive_duplicates

MEDIUM

Remove consecutive duplicate characters from a string. Example: 'aabbccaab' ⇒ 'abcab'

Answer:

```
def remove_consecutive_dups(s: str) -> str:
    if not s:
        return s
    result = [s[0]]
    for c in s[1:]:
        if c != result[-1]:
            result.append(c)
    return ''.join(result)
```

Q70. dict_from_two_lists

EASY

Create a dictionary from two lists (keys and values). Example: ['a','b'], [1,2] ⇒ {'a':1, 'b':2}

Answer:

```
def dict_from_lists(keys: list, values: list) -> dict:
    return dict(zip(keys, values))
```

Section B: Data Processing

Function-type questions involving data transformation, filtering, and aggregation.

Q71. [PYQ] is_all_same_word_twice

HARD

Check if all strings: same word repeated twice with hyphen. Example: ["fast-fast","go-go"] ⇒ True. Empty word invalid.

Answer:

```
def is_all_same_word_twice(strings: list) -> bool:
    for s in strings:
        parts = s.split("-")
        if len(parts) != 2:
            return False
        if parts[0] == "" or parts[0] != parts[1]:
            return False
    return True
```

Q72. [PYQ] most_occurring_first_letter

HARD

Find letter most frequently occurring as first letter of words in a passage. Case-insensitive, return lowercase.

Answer:

```
def most_occurring_first_letter(passage: str) -> str:
    freq = {}
    for word in passage.split():
        if word:
            letter = word[0].lower()
            freq[letter] = freq.get(letter, 0) + 1
    return max(freq, key=freq.get)
```

Q73. [PYQ] row_with_most_zeros

MEDIUM

Return row index with maximum zeros in a 2D matrix.

Answer:

```
def row_index_with_most_number_of_zeros(matrix: list) -> int:
    max_zeros = -1
    max_idx = -1
    for i, row in enumerate(matrix):
        zeros = row.count(0)
        if zeros > max_zeros:
            max_zeros = zeros
            max_idx = i
    return max_idx
```

Q74. [PYQ] top_k_teams

HARD

Given list of dicts with "name","runs","team", return top k teams by total runs descending.

Answer:

```
def top_k_teams(batsmen: list, k: int) -> list:
    team_runs = {}
    for b in batsmen:
        team = b["team"]
        team_runs[team] = team_runs.get(team, 0) + b["runs"]
    return sorted(team_runs, key=team_runs.get, reverse=True)[:k]
```

Q75. [PYQ] count_positive_ignore_none

EASY

Count positive integers ignoring None and zeros. Example: [1,-2,3,0,None,4] ⇒ 3

Answer:

```
def count_positive_ignore_none(nums: list) -> int:
    return sum(1 for n in nums if n is not None and n > 0)
```

Q76. [PYQ] walk_matrix

HARD

Walk square matrix in L/Z/O shape. L: down left col then right bottom. Z: top row, diagonal, bottom. O: clockwise border.

Answer:

```
def walk_matrix(M, shape) -> list:
    n = len(M)
    if shape == "L":
        return [M[i][0] for i in range(n)] + [M[n-1][j] for j in range(1,n)]
    elif shape == "Z":
        r = [M[0][j] for j in range(n)]
        for i in range(1,n-1): r.append(M[i][n-1-i])
        r += [M[n-1][j] for j in range(n)]
        return r
    elif shape == "O":
        r = [M[0][j] for j in range(n)]
        r += [M[i][n-1] for i in range(1,n)]
        r += [M[n-1][j] for j in range(n-2,-1,-1)]
        r += [M[i][0] for i in range(n-2,0,-1)]
        return r
```

Q77. [PYQ] get_words_by_criteria

HARD

Filter: "continuous" (ends "ing"), "vowel_rich" (>5 vowels), "consonant_rich" (>5 consonants), "sorted" (ascending). Invalid ⇒ None.

Answer:

```
def get_words_by_criteria(words, criteria=None):
    vowels = "aeiouAEIOU"
    if criteria == "continuous":
        return [w for w in words if w.lower().endswith("ing")]
    elif criteria == "vowel_rich":
        return [w for w in words if sum(1 for c in w if c in vowels) > 5]
    elif criteria == "consonant_rich":
        return [w for w in words if sum(1 for c in w if c.isalpha() and c not in vowels)
            > 5]
    elif criteria == "sorted":
        return [w for w in words if list(w.lower()) == sorted(w.lower())]
    return None
```

Q78. group_by_first_letter

MEDIUM

Group words by first letter (lowercase). Example: ["Apple","ant","Banana"] ⇒ {"a":["Apple","ant"],"b":["Banana"]}

Answer:

```
def group_by_first_letter(words: list) -> dict:
    groups = {}
    for word in words:
        key = word[0].lower()
        groups.setdefault(key, []).append(word)
    return groups
```

Q79. matrix_column_sums

MEDIUM

Return list of column sums. Example: [[1,2,3],[4,5,6]] ⇒ [5,7,9]

Answer:

```
def matrix_column_sums(matrix: list) -> list:
    cols = len(matrix[0])
    return [sum(row[j] for row in matrix) for j in range(cols)]
```

Q80. filter_by_value_range

EASY

Keep dict items where value is between lo and hi inclusive.

Answer:

```
def filter_by_value_range(d: dict, lo: int, hi: int) -> dict:
    return {k: v for k, v in d.items() if lo <= v <= hi}
```

Q81. word_frequency

MEDIUM

Count frequency of each word in a string. Example: "the cat the dog" ⇒ {"the":2,"cat":1,"dog":1}

Answer:

```python
def word_frequency(s: str) -> dict:
    freq = {}
    for word in s.split():
        freq[word] = freq.get(word, 0) + 1
    return freq
```

Q82. students_above_average

MEDIUM

Given list of (name, marks) tuples, return names of students above class average.

Answer:

```python
def students_above_average(students: list) -> list:
    avg = sum(m for _, m in students) / len(students)
    return [name for name, m in students if m > avg]
```

Q83. flatten_dict_values

MEDIUM

Given dict with list values, flatten in key-sorted order. Example: {"b":[3],"a":[1,2]} ⇒ [1,2,3]

Answer:

```python
def flatten_dict_values(d: dict) -> list:
    result = []
    for key in sorted(d):
        result.extend(d[key])
    return result
```

Q84. find_pairs_with_sum

HARD

Find all unique pairs in list that sum to target. Return list of tuples.

Answer:

```python
def find_pairs_with_sum(lst: list, target: int) -> list:
    seen = set()
    pairs = set()
    for x in lst:
        comp = target - x
        if comp in seen:
            pairs.add((min(x, comp), max(x, comp)))
        seen.add(x)
    return list(pairs)
```

Q85. sort_dict_by_value

EASY

Return dict sorted by values ascending.

Answer:
```
def sort_dict_by_value(d: dict) -> dict:
    return dict(sorted(d.items(), key=lambda x: x[1]))
```

Q86. remove_short_words

EASY

Remove words shorter than n characters from a list.

Answer:
```
def remove_short_words(words: list, n: int) -> list:
    return [w for w in words if len(w) >= n]
```

Q87. moving_average

HARD

Calculate moving average with window size k. Example: [1,2,3,4,5], k=3 ⇒ [2.0, 3.0, 4.0]

Answer:
```
def moving_average(lst: list, k: int) -> list:
    return [sum(lst[i:i+k])/k for i in range(len(lst)-k+1)]
```

Q88. count_char_types_in_list

MEDIUM

Given list of strings, return total count of digits, letters, and spaces across all strings.

Answer:
```
def count_char_types(strings: list) -> dict:
    digits = letters = spaces = 0
    for s in strings:
        for c in s:
            if c.isdigit(): digits += 1
            elif c.isalpha(): letters += 1
            elif c == " ": spaces += 1
    return {"digits": digits, "letters": letters, "spaces": spaces}
```

Q89. matrix_multiply_scalar

EASY

Multiply every element of 2D matrix by scalar. Return new matrix.

Answer:
```
def matrix_multiply_scalar(matrix: list, scalar: int) -> list:
    return [[cell * scalar for cell in row] for row in matrix]
```

Q90. longest_consecutive_sequence

HARD

Find length of longest consecutive sequence. Example: [100,4,200,1,3,2] ⇒ 4 (1,2,3,4)

Answer:
```
def longest_consecutive(lst: list) -> int:
    s = set(lst)
    best = 0
    for x in s:
        if x - 1 not in s:
            length = 1
            while x + length in s:
                length += 1
            best = max(best, length)
    return best
```

Q91. common_keys_with_same_values

MEDIUM

Return list of keys that exist in both dicts with the same value.

Answer:
```
def common_keys_same_values(d1: dict, d2: dict) -> list:
    return [k for k in d1 if k in d2 and d1[k] == d2[k]]
```

Q92. pivot_list_around_element

HARD

Rearrange so elements < pivot come first, then pivot, then elements > pivot.

Answer:
```
def pivot_list(lst: list, pivot) -> list:
    less = [x for x in lst if x < pivot]
    equal = [x for x in lst if x == pivot]
    greater = [x for x in lst if x > pivot]
    return less + equal + greater
```

Q93. count_words_starting_with_vowel

EASY

Count words that start with a vowel in a string.

Answer:
```
def count_words_starting_with_vowel(s: str) -> int:
    return sum(1 for w in s.split() if w and w[0].lower() in "aeiou")
```

Q94. inverse_mapping

MEDIUM

Given dict with list values, create inverse: {"a":[1,2],"b":[2,3]} ⇒ {1:["a"],2:["a","b"],3:["b"]}

Answer:
```
def inverse_mapping(d: dict) -> dict:
    result = {}
    for k, vals in d.items():
```

```
        for v in vals:
            result.setdefault(v, []).append(k)
    return result
```

Q95. sum_nested_list

MEDIUM

Sum all numbers in a 2D list.

Answer:

```
def sum_nested(matrix: list) -> int:
    return sum(sum(row) for row in matrix)
```

Q96. merge_sorted_lists

MEDIUM

Merge two sorted lists into one sorted list.

Answer:

```
def merge_sorted(l1: list, l2: list) -> list:
    result = []
    i = j = 0
    while i < len(l1) and j < len(l2):
        if l1[i] <= l2[j]:
            result.append(l1[i]); i += 1
        else:
            result.append(l2[j]); j += 1
    result.extend(l1[i:])
    result.extend(l2[j:])
    return result
```

Q97. top_n_values_from_dict

MEDIUM

Return list of top n keys by value from a dictionary.

Answer:

```
def top_n_keys(d: dict, n: int) -> list:
    return sorted(d, key=d.get, reverse=True)[:n]
```

Q98. zip_with_index

EASY

Return list of tuples (index, element) for each element. Example: ['a','b','c'] ⇒ [(0,'a'),(1,'b'),(2,'c')]

Answer:

```
def zip_with_index(lst: list) -> list:
    return list(enumerate(lst))
```

Q99. group_consecutive

HARD

Group consecutive equal elements. Example: [1,1,2,2,2,3,1,1] ⇒ [[1,1],[2,2,2],[3],[1,1]]

Answer:

```
def group_consecutive(lst: list) -> list:
    if not lst:
        return []
    groups = [[lst[0]]]
    for x in lst[1:]:
        if x == groups[-1][-1]:
            groups[-1].append(x)
        else:
            groups.append([x])
    return groups
```

Q100. dict_value_sum

EASY

Return sum of all values in a dictionary. Example: {'a':1,'b':2,'c':3} ⇒ 6

Answer:

```
def dict_value_sum(d: dict) -> int:
    return sum(d.values())
```

Q101. unique_elements_in_order

MEDIUM

Return unique elements preserving first occurrence order. Example: [3,1,2,1,3,4] ⇒ [3,1,2,4]

Answer:

```
def unique_in_order(lst: list) -> list:
    seen = set()
    result = []
    for x in lst:
        if x not in seen:
            seen.add(x)
            result.append(x)
    return result
```

Q102. running_max

MEDIUM

Return list of running maximums. Example: [3,1,4,1,5] ⇒ [3,3,4,4,5]

Answer:

```
def running_max(lst: list) -> list:
    result = []
    curr_max = float('-inf')
    for x in lst:
        curr_max = max(curr_max, x)
        result.append(curr_max)
```

```
    return result
```

Q103. rotate_matrix_90

HARD

Rotate a square matrix 90 degrees clockwise. Example: [[1,2],[3,4]] ⇒ [[3,1],[4,2]]

Answer:

```
def rotate_90(matrix: list) -> list:
    n = len(matrix)
    return [[matrix[n-1-j][i] for j in range(n)] for i in range(n)]
```

Q104. count_even_odd

EASY

Return a dict with counts of even and odd numbers. Example: [1,2,3,4] ⇒ {'even':2, 'odd':2}

Answer:

```
def count_even_odd(lst: list) -> dict:
    even = sum(1 for x in lst if x % 2 == 0)
    return {'even': even, 'odd': len(lst) - even}
```

Q105. nested_dict_lookup

MEDIUM

Given a nested dict and list of keys, traverse and return value. Example: {'a':{'b':1}}, ['a','b'] ⇒ 1. Return None if path invalid.

Answer:

```
def nested_lookup(d: dict, keys: list):
    curr = d
    for k in keys:
        if isinstance(curr, dict) and k in curr:
            curr = curr[k]
        else:
            return None
    return curr
```

Q106. partition_list

MEDIUM

Partition list into two lists: elements satisfying a threshold and the rest. Example: [5,2,8,1], threshold=4 ⇒ ([5,8],[2,1])

Answer:

```
def partition(lst: list, threshold: int) -> tuple:
    above = [x for x in lst if x >= threshold]
    below = [x for x in lst if x < threshold]
    return (above, below)
```

Q107. interleave_dicts

MEDIUM

Merge two dicts; if key in both, keep the sum. Example: {'a':1,'b':2}, {'b':3,'c':4} ⇒ {'a':1,'b':5,'c':4}

Answer:

```
def interleave_dicts(d1: dict, d2: dict) -> dict:
    result = dict(d1)
    for k, v in d2.items():
        result[k] = result.get(k, 0) + v
    return result
```

Q108. sliding_window_max

HARD

Return the maximum of each window of size k. Example: [1,3,2,5,4], k=3 ⇒ [3,5,5]

Answer:

```
def sliding_max(lst: list, k: int) -> list:
    return [max(lst[i:i+k]) for i in range(len(lst)-k+1)]
```

Q109. matrix_row_means

EASY

Return list of mean of each row in a 2D matrix.

Answer:

```
def row_means(matrix: list) -> list:
    return [sum(row)/len(row) for row in matrix]
```

Q110. string_pattern_check

MEDIUM

Check if a string follows a pattern like 'abab' where same letter maps to same word. Example: 'abab', 'cat dog cat dog' ⇒ True

Answer:

```
def follows_pattern(pattern: str, s: str) -> bool:
    words = s.split()
    if len(pattern) != len(words):
        return False
    p_to_w = {}
    w_to_p = {}
    for p, w in zip(pattern, words):
        if p in p_to_w and p_to_w[p] != w:
            return False
        if w in w_to_p and w_to_p[w] != p:
            return False
        p_to_w[p] = w
        w_to_p[w] = p
    return True
```

Q111. cumulative_sum

EASY

Return cumulative sum of a list. Example: [1,2,3,4] ⇒ [1,3,6,10]

Answer:

```
def cumulative_sum(lst: list) -> list:
    result = []
    s = 0
    for x in lst:
        s += x
        result.append(s)
    return result
```

Q112. matrix_trace

EASY

Return sum of diagonal elements of a square matrix. Example: [[1,2],[3,4]] ⇒ 5

Answer:

```
def trace(matrix: list) -> int:
    return sum(matrix[i][i] for i in range(len(matrix)))
```

Q113. index_of_all_occurrences

EASY

Return all indices of a target value in a list. Example: [1,2,3,2,1], target=2 ⇒ [1,3]

Answer:

```
def all_indices(lst: list, target) -> list:
    return [i for i, x in enumerate(lst) if x == target]
```

Q114. deeply_flatten

HARD

Completely flatten an arbitrarily nested list. Example: [1,[2,[3,[4]]],5] ⇒ [1,2,3,4,5]

Answer:

```
def deep_flatten(lst):
    result = []
    for item in lst:
        if isinstance(item, list):
            result.extend(deep_flatten(item))
        else:
            result.append(item)
    return result
```

Q115. dict_intersection

MEDIUM

Return dict with keys present in both dicts and values from the first. Example: {'a':1,'b':2}, {'b':3,'c':4} ⇒ {'b':2}

Answer:
```
def dict_intersection(d1: dict, d2: dict) -> dict:
    return {k: v for k, v in d1.items() if k in d2}
```

Q116. map_values

EASY

Apply a function to all values of a dict. Example: {'a':1,'b':2}, func=lambda x: x*2 ⇒ {'a':2,'b':4}

Answer:
```
def map_values(d: dict, func) -> dict:
    return {k: func(v) for k, v in d.items()}
```

Q117. count_inversions

HARD

Count the number of pairs (i,j) where i<j but lst[i]>lst[j]. Example: [2,4,1,3,5] ⇒ 3

Answer:
```
def count_inversions(lst: list) -> int:
    count = 0
    for i in range(len(lst)):
        for j in range(i+1, len(lst)):
            if lst[i] > lst[j]:
                count += 1
    return count
```

Q118. second_most_frequent

MEDIUM

Return the second most frequent element in a list. If tie, return the smaller one.

Answer:
```
def second_most_frequent(lst: list):
    freq = {}
    for x in lst:
        freq[x] = freq.get(x, 0) + 1
    sorted_items = sorted(freq.items(), key=lambda x: (-x[1], x[0]))
    return sorted_items[1][0]
```

Q119. list_to_nested_dict

HARD

Convert a list of keys and a value to a nested dict. Example: ['a','b','c'], 1 ⇒ {'a':{'b':{'c':1}}}

Answer:
```
def list_to_nested(keys: list, value):
    result = value
    for k in reversed(keys):
        result = {k: result}
    return result
```

Q120. alternating_elements

EASY

Return elements at even indices. Example: [10,20,30,40,50] ⇒ [10,30,50]

Answer:

```
def alternating(lst: list) -> list:
    return lst[::2]
```

Q121. check_subset

EASY

Check if all elements of list A are in list B. Example: [1,2], [1,2,3,4] ⇒ True

Answer:

```
def is_subset(a: list, b: list) -> bool:
    return set(a).issubset(set(b))
```

Q122. key_with_max_list_length

MEDIUM

Given dict with list values, return key with longest list. Example: {'a':[1,2],'b':[1,2,3]} ⇒ 'b'

Answer:

```
def key_max_len(d: dict):
    return max(d, key=lambda k: len(d[k]))
```

Q123. string_compression

MEDIUM

Compress string using counts. Example: 'aaabbcccc' ⇒ 'a3b2c4'. If single occurrence, use just the character.

Answer:

```
def compress(s: str) -> str:
    if not s:
        return ''
    result = []
    count = 1
    for i in range(1, len(s)):
        if s[i] == s[i-1]:
            count += 1
        else:
            result.append(s[i-1] + (str(count) if count > 1 else ''))
            count = 1
    result.append(s[-1] + (str(count) if count > 1 else ''))
    return ''.join(result)
```

Q124. product_except_self

HARD

Return list where each element is product of all others. Example: [1,2,3,4] ⇒ [24,12,8,6]. Do not use division.

Answer:

```
def product_except(lst: list) -> list:
    n = len(lst)
    result = [1] * n
    left = 1
    for i in range(n):
        result[i] *= left
        left *= lst[i]
    right = 1
    for i in range(n-1, -1, -1):
        result[i] *= right
        right *= lst[i]
    return result
```

Q125. sort_by_multiple_keys

MEDIUM

Sort list of tuples (name, age, score) by age ascending then score descending.

Answer:

```
def sort_records(records: list) -> list:
    return sorted(records, key=lambda x: (x[1], -x[2]))
```

Q126. find_duplicates

EASY

Return list of elements that appear more than once. Example: [1,2,3,2,4,3] ⇒ [2,3]

Answer:

```
def find_duplicates(lst: list) -> list:
    freq = {}
    for x in lst:
        freq[x] = freq.get(x, 0) + 1
    return [k for k, v in freq.items() if v > 1]
```

Q127. pairwise_sum

EASY

Return list of pairwise sums of adjacent elements. Example: [1,2,3,4] ⇒ [3,5,7]

Answer:

```
def pairwise_sum(lst: list) -> list:
    return [lst[i] + lst[i+1] for i in range(len(lst)-1)]
```

Q128. caesar_encrypt

MEDIUM

Encrypt a string using Caesar cipher with shift k. Only shift lowercase letters, leave rest unchanged.

Answer:

```
def caesar_encrypt(s: str, k: int) -> str:
    result = []
    for c in s:
        if c.islower():
            result.append(chr((ord(c) - ord('a') + k) % 26 + ord('a')))
        else:
            result.append(c)
    return ''.join(result)
```

Q129. max_difference_pair

MEDIUM

Return max difference between any two elements in a list. Example: [2,7,1,9] ⇒ 8

Answer:

```
def max_difference(lst: list) -> int:
    return max(lst) - min(lst)
```

Q130. swap_keys_values

EASY

Swap keys and values of a dictionary. Assume all values are unique. Example: {'a':1,'b':2} ⇒ {1:'a',2:'b'}

Answer:

```
def swap_kv(d: dict) -> dict:
    return {v: k for k, v in d.items()}
```

Section C: I/O Based Problem Solving

Full code required. Use `input()` and `print()`.

Q131. [PYQ] Run-Length Encoding

HARD

Given n positive integers, print run-length encoding. Example: 44455544445 ⇒ 3 4 3 5 4 4 1 5

Answer:

```
n = int(input())
for _ in range(n):
    s = input().strip()
    result = []
    count = 1
    for i in range(1, len(s)):
        if s[i] == s[i-1]:
            count += 1
        else:
            result.append(f"{count} {s[i-1]}")
            count = 1
    result.append(f"{count} {s[-1]}")
    print(" ".join(result))
```

Q132. [PYQ] Antakshari Chain

HARD

Find longest consecutive chain where last letter of word = first letter of next (case-insensitive).

Answer:

```
n = int(input())
for _ in range(n):
    words = input().strip().split(',')
    max_len = curr = 1
    for i in range(1, len(words)):
        if words[i-1][-1].lower() == words[i][0].lower():
            curr += 1
            max_len = max(max_len, curr)
        else:
            curr = 1
    print(max_len)
```

Q133. [PYQ] Sum/Diff Odd/Even Pairs

MEDIUM

n pairs of comma-separated ints. Print sum for odd-numbered pairs, abs diff for even.

Answer:

```
n = int(input())
for i in range(1, n+1):
    a, b = map(int, input().split(","))
```

```
    if i % 2 == 1:
        print(a + b)
    else:
        print(abs(a - b))
```

Q134. [PYQ] Swap and Reverse Lines

HARD

n lines: swap consecutive pairs, then reverse odd-positioned lines (1-indexed).

Answer:

```
n = int(input())
lines = [input().strip() for _ in range(n)]
for i in range(0, n-1, 2):
    lines[i], lines[i+1] = lines[i+1], lines[i]
for i in range(n):
    if (i+1) % 2 == 1:
        print(lines[i][::-1])
    else:
        print(lines[i])
```

Q135. Staircase Pattern

EASY

Print right-aligned staircase with # characters, n steps.

```
# n=4
   #
  ##
 ###
####
```

Answer:

```
n = int(input())
for i in range(1, n+1):
    print(" " * (n - i) + "#" * i)
```

Q136. Caesar Cipher

HARD

Shift each letter by k positions, wrap around. Non-alpha unchanged. Example: "Hello",k=3 ⇒ "Khoor"

Answer:

```
s = input()
k = int(input())
result = ""
for ch in s:
    if ch.isalpha():
        base = ord("A") if ch.isupper() else ord("a")
        result += chr((ord(ch) - base + k) % 26 + base)
    else:
```

```
        result += ch
print(result)
```

Q137. Student Grade Classifier

MEDIUM

Read n lines name,marks. Print name:grade (A≥90, B≥75, C≥60, D≥40, F). Print average.

Answer:

```
n = int(input())
total = 0
for _ in range(n):
    name, marks = input().split(",")
    m = int(marks)
    total += m
    if m >= 90: g = "A"
    elif m >= 75: g = "B"
    elif m >= 60: g = "C"
    elif m >= 40: g = "D"
    else: g = "F"
    print(f"{name}: {g}")
print(f"Average: {total/n:.2f}")
```

Q138. Matrix Transpose (I/O)

MEDIUM

Read n x m matrix, print transpose.

Answer:

```
n, m = map(int, input().split())
matrix = [list(map(int, input().split())) for _ in range(n)]
for j in range(m):
    print(' '.join(str(matrix[i][j]) for i in range(n)))
```

Q139. Frequency Counter

MEDIUM

Read n words, print each unique word with frequency, sorted by frequency descending.

Answer:

```
n = int(input())
freq = {}
for _ in range(n):
    w = input().strip()
    freq[w] = freq.get(w, 0) + 1
for w in sorted(freq, key=freq.get, reverse=True):
    print(f'{w} {freq[w]}')
```

Q140. Palindrome Checker

EASY

Read n strings. Print YES/NO for palindrome check.

Answer:

```
n = int(input())
for _ in range(n):
    s = input().strip()
    print('YES' if s == s[::-1] else 'NO')
```

Q141. Diamond Pattern

MEDIUM

Print diamond pattern with n rows (upper half). Use * characters.

```
# n=3
  *
 ***
*****
 ***
  *
```

Answer:

```
n = int(input())
for i in range(n):
    print(" " * (n-1-i) + "*" * (2*i+1))
for i in range(n-2, -1, -1):
    print(" " * (n-1-i) + "*" * (2*i+1))
```

Q142. Number Triangle

EASY

Print number triangle with n rows.

```
# n=4
1
1 2
1 2 3
1 2 3 4
```

Answer:

```
n = int(input())
for i in range(1, n+1):
    print(' '.join(str(j) for j in range(1, i+1)))
```

Q143. Sum of Digits (I/O)

EASY

Read n integers, print sum of digits for each.

Answer:

```
n = int(input())
for _ in range(n):
    num = input().strip()
    print(sum(int(d) for d in num if d.isdigit()))
```

Q144. Fibonacci Sequence

EASY

Print first n Fibonacci numbers separated by spaces.

Answer:

```
n = int(input())
a, b = 0, 1
result = []
for _ in range(n):
    result.append(str(a))
    a, b = b, a + b
print(' '.join(result))
```

Q145. Vowel Remover

EASY

Read n strings, print each with vowels removed.

Answer:

```
n = int(input())
for _ in range(n):
    s = input()
    print("".join(c for c in s if c.lower() not in "aeiou"))
```

Q146. Two Sum Indices

MEDIUM

Given n numbers and target, print indices of two numbers that sum to target.

Answer:

```
n = int(input())
nums = list(map(int, input().split()))
target = int(input())
seen = {}
for i, x in enumerate(nums):
    comp = target - x
    if comp in seen:
        print(seen[comp], i)
        break
    seen[x] = i
```

Q147. Longest Word Per Line

EASY

Read n lines, print the longest word from each line.

Answer:

```
n = int(input())
for _ in range(n):
    words = input().split()
    print(max(words, key=len))
```

Q148. Duplicate Remover

EASY

Read n numbers, print unique numbers preserving first occurrence order.

Answer:

```
n = int(input())
nums = list(map(int, input().split()))
seen = set()
result = []
for x in nums:
    if x not in seen:
        seen.add(x)
        result.append(x)
print(' '.join(map(str, result)))
```

Q149. Matrix Row Sum

EASY

Read n x m matrix, print sum of each row.

Answer:

```
n, m = map(int, input().split())
for _ in range(n):
    row = list(map(int, input().split()))
    print(sum(row))
```

Q150. Reverse Each Word

EASY

Read n lines, reverse each word individually. Example: "hello world" ⇒ "olleh dlrow"

Answer:

```
n = int(input())
for _ in range(n):
    words = input().split()
    print(' '.join(w[::-1] for w in words))
```

Q151. Number to Words

MEDIUM

Read an integer (0-9999), print in words. Example: 123 ⇒ one two three

Answer:

```
w = {"0":"zero","1":"one","2":"two","3":"three","4":"four",
     "5":"five","6":"six","7":"seven","8":"eight","9":"nine"}
num = input().strip()
print(" ".join(w[d] for d in num))
```

Q152. Check Sorted Array

EASY

Read n numbers, print SORTED or NOT SORTED.

Answer:

```
n = int(input())
nums = list(map(int, input().split()))
if all(nums[i] <= nums[i+1] for i in range(n-1)):
    print('SORTED')
else:
    print('NOT SORTED')
```

Q153. Count Lines with Keyword

EASY

Read n lines and keyword. Print count of lines containing that keyword.

Answer:

```
n = int(input())
keyword = input().strip()
count = 0
for _ in range(n):
    line = input()
    if keyword in line:
        count += 1
print(count)
```

Q154. Spiral First Row/Col

MEDIUM

Read n x n matrix. Print first row left-to-right, last column top-to-bottom, last row right-to-left, first column bottom-to-top.

Answer:

```
n = int(input())
M = [list(map(int, input().split())) for _ in range(n)]
result = [M[0][j] for j in range(n)]
result += [M[i][n-1] for i in range(1, n)]
result += [M[n-1][j] for j in range(n-2, -1, -1)]
result += [M[i][0] for i in range(n-2, 0, -1)]
print(' '.join(map(str, result)))
```

Q155. Zigzag Pattern

MEDIUM

Print zigzag of * for n rows.

```
# n=3, width=7
*     *
 * * *
  *
```

Answer:

```
n = int(input())
w = 2*n - 1
for i in range(n):
    row = [" "] * w
    row[i] = "*"
    row[w-1-i] = "*"
    print("".join(row))
```

Q156. Bracket Validator

HARD

Read n strings of brackets. Print VALID or INVALID for each.

Answer:

```
n = int(input())
for _ in range(n):
    s = input().strip()
    stack = []
    valid = True
    match = {")":"(", "]":"[", "}":"{"}
    for ch in s:
        if ch in "([{":
            stack.append(ch)
        elif ch in ")]}":
            if not stack or stack[-1] != match[ch]:
                valid = False
                break
            stack.pop()
    if stack: valid = False
    print("VALID" if valid else "INVALID")
```

Q157. Prime Number Check

EASY

Read n numbers, print PRIME or NOT PRIME for each.

Answer:

```
n = int(input())
for _ in range(n):
    num = int(input())
    if num < 2:
        print('NOT PRIME')
```

```
    else:
        is_p = True
        for i in range(2, int(num**0.5)+1):
            if num % i == 0:
                is_p = False
                break
        print('PRIME' if is_p else 'NOT PRIME')
```

Q158. Inverted Triangle

EASY

Print inverted triangle with * characters.

```
# n=4
****
 ***
  **
   *
```

Answer:

```
n = int(input())
for i in range(n):
    print(" " * i + "*" * (n - i))
```

Q159. Column Max

MEDIUM

Read n x m matrix, print max of each column.

Answer:

```
n, m = map(int, input().split())
matrix = [list(map(int, input().split())) for _ in range(n)]
for j in range(m):
    print(max(matrix[i][j] for i in range(n)), end=' ')
print()
```

Q160. Alternate Case

EASY

Read string, alternate upper/lower per character. Example: "hello" ⇒ "HeLlO"

Answer:

```
s = input()
result = ""
for i, ch in enumerate(s):
    result += ch.upper() if i % 2 == 0 else ch.lower()
print(result)
```

Q161. GCD of Array

MEDIUM

Read n numbers, print GCD of all.

Answer:

```
import math
n = int(input())
nums = list(map(int, input().split()))
result = nums[0]
for i in range(1, n):
    result = math.gcd(result, nums[i])
print(result)
```

Q162. Hollow Rectangle

EASY

Print hollow rectangle of * with r rows and c columns.

Answer:

```
r, c = map(int, input().split())
for i in range(r):
    if i == 0 or i == r - 1:
        print("*" * c)
    else:
        print("*" + " " * (c - 2) + "*")
```

Q163. Pascal's Triangle Row

MEDIUM

Print the nth row of Pascal's Triangle.

Answer:

```
n = int(input())
row = [1]
for i in range(1, n):
    row.append(row[-1] * (n - i) // i)
print(' '.join(map(str, row)))
```

Q164. Word Reversal in Sentence

EASY

Read a sentence, reverse word order. Example: `hello world` ⇒ `world hello`

Answer:

```
s = input()
print(' '.join(s.split()[::-1]))
```

Q165. Binary Representation

EASY

Read n integers, print binary representation of each (without 0b prefix).

Answer:
```
n = int(input())
for _ in range(n):
    num = int(input())
    print(bin(num)[2:])
```

Q166. Matrix Diagonal Difference

MEDIUM

Read n x n matrix. Print absolute difference between primary and secondary diagonal sums.

Answer:
```
n = int(input())
M = [list(map(int, input().split())) for _ in range(n)]
primary = sum(M[i][i] for i in range(n))
secondary = sum(M[i][n-1-i] for i in range(n))
print(abs(primary - secondary))
```

Q167. String Pattern Matching

MEDIUM

Read n strings and a pattern. Print all strings containing the pattern.

Answer:
```
n = int(input())
pattern = input().strip()
for _ in range(n):
    s = input().strip()
    if pattern in s:
        print(s)
```

Q168. Power Calculator

EASY

Read n lines of base,exp. Print base^exp for each.

Answer:
```
n = int(input())
for _ in range(n):
    b, e = map(int, input().split(","))
    print(b ** e)
```

Q169. Sandwich Number

MEDIUM

Read n. For each number, check if middle digit is average of first and last. Print YES/NO.

Answer:
```
n = int(input())
for _ in range(n):
    s = input().strip()
```

```
    if len(s) == 3:
        f, m, l = int(s[0]), int(s[1]), int(s[2])
        print('YES' if m == (f + l) / 2 else 'NO')
    else:
        print('NO')
```

Q170. Cumulative Product

EASY

Read n numbers, print cumulative product.

Answer:

```
n = int(input())
nums = list(map(int, input().split()))
prod = 1
result = []
for x in nums:
    prod *= x
    result.append(str(prod))
print(' '.join(result))
```

Q171. Border Elements

MEDIUM

Read n x m matrix. Print only the border elements in clockwise order.

Answer:

```
n, m = map(int, input().split())
M = [list(map(int, input().split())) for _ in range(n)]
result = list(M[0])
for i in range(1, n): result.append(M[i][m-1])
for j in range(m-2, -1, -1): result.append(M[n-1][j])
for i in range(n-2, 0, -1): result.append(M[i][0])
print(' '.join(map(str, result)))
```

Q172. Largest of Three

EASY

Read three integers and print the largest.

Answer:

```
a, b, c = map(int, input().split())
print(max(a, b, c))
```

Q173. Multiplication Table

EASY

Read integer n. Print multiplication table from 1 to 10.

Answer:

```
n = int(input())
for i in range(1, 11):
    print(f'{n} x {i} = {n*i}')
```

Q174. Even/Odd Counter

EASY

Read n integers. Print count of even and odd numbers.

Answer:

```
n = int(input())
nums = list(map(int, input().split()))
even = sum(1 for x in nums if x % 2 == 0)
print(f'Even: {even}')
print(f'Odd: {n - even}')
```

Q175. String Rotation Check

MEDIUM

Read two strings. Check if one is a rotation of the other. Example: 'abcde' and 'cdeab' ⇒ Yes

Answer:

```
s1 = input().strip()
s2 = input().strip()
if len(s1) == len(s2) and s2 in s1 + s1:
    print('Yes')
else:
    print('No')
```

Q176. Right Triangle Pattern

EASY

Read n. Print right triangle of stars. Row i has i stars.

Answer:

```
n = int(input())
for i in range(1, n+1):
    print('*' * i)
```

Q177. Matrix Addition

MEDIUM

Read two n x m matrices. Print their sum matrix.

Answer:

```
n, m = map(int, input().split())
A = [list(map(int, input().split())) for _ in range(n)]
B = [list(map(int, input().split())) for _ in range(n)]
for i in range(n):
    print(' '.join(str(A[i][j]+B[i][j]) for j in range(m)))
```

Q178. Armstrong Number

MEDIUM

Read n. Check if it is an Armstrong number (sum of digits raised to power of number of digits equals the number). Print Yes or No.

Answer:

```
n = input().strip()
num_digits = len(n)
total = sum(int(d)**num_digits for d in n)
print('Yes' if total == int(n) else 'No')
```

Q179. Count Substring

EASY

Read main string and substring. Print how many times substring appears.

Answer:

```
s = input().strip()
sub = input().strip()
count = 0
for i in range(len(s) - len(sub) + 1):
    if s[i:i+len(sub)] == sub:
        count += 1
print(count)
```

Q180. Perfect Number

MEDIUM

Read n. Check if it is a perfect number (sum of proper divisors equals n). Print Yes or No.

Answer:

```
n = int(input())
divisors_sum = sum(i for i in range(1, n) if n % i == 0)
print('Yes' if divisors_sum == n else 'No')
```

Q181. Sort Words Alphabetically

EASY

Read a line of space-separated words. Print them sorted alphabetically.

Answer:

```
words = input().split()
words.sort()
print(' '.join(words))
```

Q182. Remove Vowels

EASY

Read a string. Print it with all vowels removed.

Answer:

```
s = input().strip()
print(''.join(c for c in s if c.lower() not in 'aeiou'))
```

Q183. LCM of Two Numbers

MEDIUM

Read two integers. Print their LCM.

Answer:

```
import math
a, b = map(int, input().split())
print(a * b // math.gcd(a, b))
```

Q184. Floyd's Triangle

EASY

Read n. Print Floyd's triangle (consecutive numbers filling rows).

Answer:

```
n = int(input())
num = 1
for i in range(1, n+1):
    row = []
    for j in range(i):
        row.append(str(num))
        num += 1
    print(' '.join(row))
```

Q185. Unique Characters

EASY

Read a string. Print all unique characters in order of first appearance.

Answer:

```
s = input().strip()
seen = set()
result = []
for c in s:
    if c not in seen:
        seen.add(c)
        result.append(c)
print(''.join(result))
```

Q186. Sum of Series

EASY

Read n. Print sum of series: 1 + 1/2 + 1/3 + ... + 1/n (rounded to 2 decimal places).

Answer:

```
n = int(input())
result = sum(1/i for i in range(1, n+1))
print(f'{result:.2f}')
```

Q187. Matrix Identity Check

MEDIUM

Read n x n matrix. Check if it is an identity matrix. Print Yes or No.

Answer:

```
n = int(input())
M = [list(map(int, input().split())) for _ in range(n)]
is_identity = True
for i in range(n):
    for j in range(n):
        if i == j and M[i][j] != 1:
            is_identity = False
        if i != j and M[i][j] != 0:
            is_identity = False
print('Yes' if is_identity else 'No')
```

Q188. Second Largest in List

EASY

Read n integers. Print the second largest.

Answer:

```
n = int(input())
nums = list(map(int, input().split()))
unique = sorted(set(nums))
print(unique[-2])
```

Q189. Palindrome String

EASY

Read a string. Print Yes if it is a palindrome (case-insensitive), No otherwise.

Answer:

```
s = input().strip().lower()
print('Yes' if s == s[::-1] else 'No')
```

Q190. Character Frequency Map

MEDIUM

Read a string. Print each character and its frequency, one per line, in order of first appearance.

Answer:

```
s = input().strip()
freq = {}
order = []
```

```
for c in s:
    if c not in freq:
        freq[c] = 0
        order.append(c)
    freq[c] += 1
for c in order:
    print(f'{c}: {freq[c]}')
```

Q191. Number Pyramid

MEDIUM

Read n. Print a centered number pyramid. Row i has numbers 1 to i.

Answer:

```
n = int(input())
for i in range(1, n+1):
    spaces = ' ' * (n - i)
    nums = ' '.join(str(j) for j in range(1, i+1))
    print(spaces + nums)
```

Q192. Leap Year Checker

EASY

Read a year. Print 'Leap' if it is a leap year, 'Not Leap' otherwise.

Answer:

```
y = int(input())
if (y % 4 == 0 and y % 100 != 0) or y % 400 == 0:
    print('Leap')
else:
    print('Not Leap')
```

Q193. Merge Two Sorted Arrays (I/O)

MEDIUM

Read two sorted arrays (n then elements, m then elements). Print merged sorted array.

Answer:

```
n = int(input())
a = list(map(int, input().split()))
m = int(input())
b = list(map(int, input().split()))
i = j = 0
result = []
while i < n and j < m:
    if a[i] <= b[j]:
        result.append(a[i]); i += 1
    else:
        result.append(b[j]); j += 1
result.extend(a[i:])
result.extend(b[j:])
```

```
print(' '.join(map(str, result)))
```

Q194. Sum of Prime Numbers

MEDIUM

Read n. Print sum of all prime numbers up to n.

Answer:

```
n = int(input())
def is_prime(x):
    if x < 2: return False
    for i in range(2, int(x**0.5)+1):
        if x % i == 0: return False
    return True
print(sum(i for i in range(2, n+1) if is_prime(i)))
```

Q195. Hourglass Pattern

MEDIUM

Read n (odd). Print an hourglass pattern of stars.

Answer:

```
n = int(input())
for i in range(n, 0, -2):
    spaces = ' ' * ((n - i) // 2)
    print(spaces + '* ' * i)
for i in range(3, n+1, 2):
    spaces = ' ' * ((n - i) // 2)
    print(spaces + '* ' * i)
```

Q196. Digit Sum Until Single

EASY

Read integer n. Repeatedly sum its digits until a single digit remains. Print the result.

Answer:

```
n = int(input())
while n >= 10:
    n = sum(int(d) for d in str(n))
print(n)
```

Q197. Reverse a Number

EASY

Read an integer. Print it reversed. Example: 1234 ⇒ 4321.

Answer:

```
n = input().strip()
if n[0] == '-':
    print('-' + n[:0:-1])
else:
```

```
    print(n[::-1])
```

Q198. Common Elements of Two Lists

EASY

Read two lines of space-separated integers. Print common elements in sorted order.

Answer:

```
a = set(map(int, input().split()))
b = set(map(int, input().split()))
print(' '.join(map(str, sorted(a & b))))
```

Q199. Matrix Spiral Print

HARD

Read n x m matrix. Print elements in spiral order.

Answer:

```
n, m = map(int, input().split())
M = [list(map(int, input().split())) for _ in range(n)]
result = []
top, bottom, left, right = 0, n-1, 0, m-1
while top <= bottom and left <= right:
    for j in range(left, right+1): result.append(M[top][j])
    top += 1
    for i in range(top, bottom+1): result.append(M[i][right])
    right -= 1
    if top <= bottom:
        for j in range(right, left-1, -1): result.append(M[bottom][j])
        bottom -= 1
    if left <= right:
        for i in range(bottom, top-1, -1): result.append(M[i][left])
        left += 1
print(' '.join(map(str, result)))
```

Q200. Temperature Converter

EASY

Read n temperatures in Celsius. Print each converted to Fahrenheit (rounded to 1 decimal).

Answer:

```
n = int(input())
for _ in range(n):
    c = float(input())
    print(f'{c * 9/5 + 32:.1f}')
```